

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Высшая школа программной инженерии

Работа допущена к защите

Директор ВШПИ

_____ П.Д.Дробинцев

« ____ » _____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

работа бакалавра

ОПТИМИЗАЦИЯ КОНФИГУРАЦИИ POSTGRESQL НА ОСНОВЕ АЛГОРИТМА SMAC

по направлению подготовки (специальности)

09.03.04 Программная инженерия

Направленность (профиль)

**09.03.04_01 Технология разработки и сопровождения качественного
программного продукта**

Выполнил студент гр.

5130904/20104

Р.Р.Ганиуллин

Руководитель

старший преподаватель

О.В.Прокофьев

Консультант

по нормоконтролю

Е.Г.Локшина

Санкт-Петербург
2026

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

УТВЕРЖДАЮ

Директор ВШПИ

П. Д. Дробинцев

«__» _____ 2026 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту

Ганиуллину Родиону Ринатовичу 5130904/20104

фамилия, имя, отчество (при наличии), номер группы

1. Тема работы: Оптимизация конфигурации PostgreSQL на основе алгоритма SMAC

2. Срок сдачи студентом законченной работы: 18.05.2026

3. Исходные данные по работе:

Для улучшения производительности работы баз данных необходимо производить тонкую подстройку ее конфигурации под конкретную нагрузку. Требуется разработать алгоритм автоматизированного тюнинга конфигурации баз данных на основе алгоритмов Байесовской оптимизации

4. Содержание работы (перечень подлежащих разработке вопросов):

- 4.1. Анализ предметной области
- 4.2. Анализ алгоритмов Байесовской оптимизации
- 4.3. Разработка архитектуры системы
- 4.4. Реализация на языке Python
- 4.5. Проведение экспериментов с различными типами нагрузки

5. Перечень графического материала (с указанием обязательных чертежей):

6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):

Python (python.org), Optuna (optuna.org), SMAC3 (github.com/automl/SMAC3),
Prometheus (prometheus.io), PostgreSQL (postgresql.org),
pgbench (postgresql.org/docs/current/pgbench.html), pgTune (pgtune.leopard.in.ua),
docker (docker.com), postgres_exporter (github.com/prometheus-community/postgres_exporter),
pg_stat_statements (postgresql.org/docs/current/pgstatstatements.html)

7. Консультанты по работе:

8. Дата выдачи задания: 17.03.2026

Руководитель ВКР _____
(подпись) О.В.Прокофьев
инициалы, фамилия

Задание принял к исполнению 17.03.2026

Студент _____
(подпись) Р.Р.Ганиуллин
инициалы, фамилия

РЕФЕРАТ

Отчёт 54 с., 9 рис., 12 табл., 13 ист.

POSTGRESQL, АВТОМАТИЧЕСКАЯ НАСТРОЙКА СУБД, БАЙЕСОВСКАЯ ОПТИМИЗАЦИЯ, SMAC, TPE, PGBENCH, КОНФИГУРАЦИОННЫЕ ПАРАМЕТРЫ

Выпускная квалификационная работа посвящена разработке прототипа системы автоматической оптимизации конфигурационных параметров СУБД PostgreSQL. Предметом исследования являются методы автоматической настройки параметров СУБД с использованием байесовской оптимизации.

В работе проведён анализ существующих подходов к настройке PostgreSQL, выявлены их ключевые недостатки. Разработана архитектура системы, включающая оптимизатор на основе фреймворка Optuna с сэмплерами SMAC и TPE, агент управления конфигурацией, генератор нагрузки pgbench и систему сбора метрик Prometheus. Сформировано пространство поиска из 25 конфигурационных параметров. Проведены эксперименты на трёх сценариях нагрузки: OLTP (TPS), OLTP (задержка) и OLAP (TPS), по три повторных запуска каждым сэмплером. SMAC достиг прироста +14,3% TPS на OLTP-нагрузке и снижения задержки на 31,2% относительно конфигурации pgTune; TPE показал сопоставимые результаты.

Результаты подтверждают применимость байесовской оптимизации для задач автоматической настройки СУБД и демонстрируют возможность создания открытого инструмента, превосходящего эвристические методы без участия эксперта.

ABSTRACT

Work: 54 p., 9 fig., 12 tab., 13 ref.

POSTGRESQL, AUTOMATED DBMS CONFIGURATION TUNING,
BAYESIAN OPTIMIZATION, SMAC, TPE, PGBENCH, CONFIGURATION
PARAMETERS

This thesis presents a prototype system for automated optimization of PostgreSQL DBMS configuration parameters using Bayesian optimization. The subject of study is automated database tuning methods based on black-box optimization.

Existing approaches to PostgreSQL tuning are analyzed, and their key shortcomings are identified. The proposed system architecture comprises an Optuna-based optimizer with SMAC and TPE samplers, a configuration management agent, a pgbench load generator, and a Prometheus metrics collection system. A search space of 25 configuration parameters was defined. Experiments were conducted on three workload scenarios — OLTP (TPS), OLTP (latency), and OLAP (TPS) — with three independent runs per sampler. SMAC achieved a +14.3% TPS improvement on OLTP workloads and a 31.2% latency reduction relative to the pgTune baseline; TPE showed comparable results.

The results confirm the applicability of Bayesian optimization for automated DBMS tuning and demonstrate the feasibility of building an open-source tool that outperforms heuristic methods without expert involvement.

СОДЕРЖАНИЕ

Перечень сокращений и обозначений	9
Введение	11
1 Анализ предметной области	14
1.1 Подходы к настройке СУБД	14
1.1.1 Ручная настройка	14
1.1.2 Эвристические методы	14
1.1.3 Алгоритмические методы	14
1.1.4 Методы машинного обучения	15
1.2 Анализ существующих инструментов	16
1.2.1 pg_stat_statements	16
1.2.2 PgTune	16
1.2.3 OtterTune	17
1.2.4 Другие решения	17
1.3 Сравнительный анализ и выявление недостатков	18
1.4 Требования к разрабатываемой системе	18
1.5 Выводы по главе	19
2 Байесовская оптимизация как метод настройки СУБД	21
2.1 Постановка задачи оптимизации конфигурации	21
2.2 Сравнение подходов к автоматическому поиску конфигурации . .	21
2.3 Байесовская оптимизация	23
2.4 Суррогатные модели и функции приобретения	24
2.4.1 Суррогатная модель	24
2.4.2 Функция приобретения	25
2.5 Алгоритм TPE	26
2.6 Алгоритм SMAC	26

2.7	Пространство поиска конфигурационных параметров	27
2.7.1	Параметры памяти	27
2.7.2	Параметры WAL и контрольных точек	28
2.7.3	Параметры планировщика и ввода-вывода	28
2.7.4	Параметры фонового записывателя	28
2.7.5	Параметры соединений и параллелизма	29
2.8	Метрики оценки производительности	29
2.9	Сравнение подходов к байесовской оптимизации	30
2.10	Выводы по главе	31
3	Архитектура и реализация системы	32
3.1	Общая архитектура системы	32
3.2	Инфраструктура развёртывания	33
3.3	Изоляция тестовой среды	34
3.4	Управление конфигурацией	35
3.5	Цикл одной итерации оптимизации	36
3.6	Профили нагрузки	37
3.7	Базовая конфигурация (pgTune baseline)	38
3.8	Хранение результатов и визуализация	38
3.9	Реализация	40
3.10	Средства разработки и контроля качества	40
3.10.1	Контроль версий	40
3.10.2	Непрерывная интеграция	41
3.10.3	Форма представления результатов	41
3.11	Тестирование	41
3.12	Выводы по главе	42
4	Апробация и анализ результатов	43
4.1	Методика проведения экспериментов	43

4.2	Тестовая среда	43
4.3	Базовые конфигурации	44
4.4	Результаты экспериментов	45
4.4.1	Прирост производительности	45
4.4.2	Динамика сходимости	45
4.4.3	Важность параметров	46
4.4.4	Коэффициент попаданий в буферный кеш	47
4.4.5	Характеристики лучших найденных конфигураций	47
4.5	Анализ результатов	48
4.5.1	Сравнение сэмплеров	48
4.5.2	Влияние отдельных параметров	49
4.5.3	Оценка погрешности измерений	49
4.5.4	Ограничения и применимость	50
4.6	Выводы по главе	50
	Заключение	52
	Список использованных источников	54

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

СУБД — система управления базами данных.

БД — база данных.

OLTP (Online Transaction Processing) — обработка транзакций в реальном времени; рабочая нагрузка, характеризующаяся большим числом коротких транзакций.

OLAP (Online Analytical Processing) — аналитическая обработка данных; рабочая нагрузка, характеризующаяся сложными запросами к большим объёмам данных.

TPS (Transactions Per Second) — количество транзакций в секунду; метрика пропускной способности СУБД.

WAL (Write-Ahead Log) — журнал предзаписи; механизм обеспечения надёжности в PostgreSQL.

БО — байесовская оптимизация.

TPE (Tree-structured Parzen Estimator) — древовидный оценщик Парзена; алгоритм байесовской оптимизации.

SMAC (Sequential Model-based Algorithm Configuration) — алгоритм последовательной конфигурации на основе модели.

ВКР — выпускная квалификационная работа.

pgTune — утилита автоматической генерации рекомендаций по настройке PostgreSQL на основе характеристик сервера.

pgbench — встроенный в PostgreSQL инструмент нагрузочного тестирования.

GP (Gaussian Process) — гауссовский процесс; вероятностная модель, применяемая в качестве суррогата в классических алгоритмах байесовской оптимизации.

RF (Random Forest) — случайный лес; ансамблевый алгоритм машинного обучения, используемый в качестве суррогатной модели в алгоритме SMAC.

EI (Expected Improvement) — ожидаемое улучшение; функция приобретения байесовской оптимизации.

fANOVA (functional ANalysis Of VAriance) — метод анализа важности гиперпараметров на основе случайного леса.

CI (Continuous Integration) — непрерывная интеграция; практика автоматической сборки и тестирования кода при каждом изменении.

CPU (Central Processing Unit) — центральный процессор.

RAM (Random Access Memory) — оперативная память.

SSD (Solid-State Drive) — твердотельный накопитель.

NVMe (Non-Volatile Memory Express) — интерфейс высокоскоростного подключения SSD через шину PCIe.

ВВЕДЕНИЕ

Актуальность темы. Современные информационные системы сталкиваются с постоянно растущим объёмом обрабатываемых данных. В таких системах база данных нередко оказывается узким местом, определяющим общую производительность. PostgreSQL — одна из наиболее распространённых реляционных СУБД с открытым исходным кодом — предоставляет обширный набор конфигурационных параметров, влияющих на использование памяти, дисковый ввод-вывод, параллелизм и планировщик запросов.

Производительность PostgreSQL существенно зависит от того, насколько точно настройка соответствует профилю рабочей нагрузки конкретного приложения. Ручная подстройка требует глубоких знаний внутреннего устройства СУБД, значительных временных затрат на проведение экспериментов и остаётся недоступной для большинства разработчиков и администраторов. Существующие автоматические инструменты либо опираются на статические эвристики и не учитывают специфику конкретной нагрузки, либо являются проприетарными или давно не поддерживаемыми решениями.

Применение методов байесовской оптимизации позволяет рассматривать задачу настройки СУБД как задачу оптимизации дорогостоящей чёрноящичной функции и находить конфигурации, превосходящие эвристические рекомендации, без участия эксперта.

Классические алгоритмические подходы к оптимизации конфигурации — поиск по сетке (grid search) и случайный поиск (random search) — не справляются с пространствами высокой размерности при дорогостоящей целевой функции.

Поиск по сетке дискретизирует каждый параметр и перебирает все комбинации значений. При d параметрах с k вариантами каждого число конфигураций растёт как k^d : для 25 параметров и 10 значений это 10^{25} вычислений, каждое из которых занимает несколько минут. Практическое применение метода оказывается невозможным. Кроме того, поиск по сетке тратит вычислительный бюджет равномерно по всему пространству, не концентрируя усилия в перспективных областях.

Случайный поиск устраняет проблему экспоненциального взрыва: конфигурации выбираются случайно из пространства поиска [1]. Бергстра

и Бенджио показали, что при одинаковом бюджете итераций случайный поиск нередко находит лучшее решение, чем поиск по сетке, за счёт равномерного покрытия маргинальных распределений параметров. Однако и он не использует информацию о ранее проверенных конфигурациях: каждая следующая точка выбирается независимо от предыдущих результатов.

Байесовская оптимизация преодолевает это ограничение: на каждой итерации строится вероятностная суррогатная модель зависимости целевой метрики от конфигурации, и следующая точка выбирается так, чтобы максимизировать ожидаемое улучшение с учётом всех накопленных наблюдений. Это позволяет находить хорошие конфигурации за десятки итераций вместо тысяч и делает метод пригодным для задач с дорогостоящей целевой функцией.

Цель работы: разработка прототипа системы автоматической оптимизации конфигурационных параметров СУБД PostgreSQL на основе метрик производительности с применением байесовской оптимизации.

Задачи работы:

1. провести анализ существующих методов и инструментов автоматической настройки PostgreSQL, выявить их достоинства и недостатки;
2. определить и обосновать набор конфигурационных параметров, оказывающих наибольшее влияние на производительность, сформировать пространство поиска;
3. разработать архитектуру системы автоматической оптимизации конфигурации, включающей оптимизатор, агент управления конфигурацией, генератор нагрузки и систему сбора метрик;
4. реализовать прототип системы на языке Python с использованием фреймворка Optuna;
5. провести экспериментальную оценку прототипа на нагрузках типа OLTP и OLAP и сравнить результаты с базовыми конфигурациями.

Объект исследования — процесс настройки конфигурационных параметров СУБД PostgreSQL.

Предмет исследования — методы автоматической оптимизации параметров СУБД с использованием байесовской оптимизации.

Методы и инструменты исследования. В работе применяются методы системного анализа и экспериментального моделирования. Для оптимизации конфигурации используется фреймворк Optuna [2] с алгоритмами TPE [3] и SMAC [4]. Нагрузочное тестирование осуществляется инструментом pgbench [5]. Метрики производительности собираются системой мониторинга Prometheus [6] через экспортёр postgres-exporter.

Научная значимость работы заключается в применении современных методов байесовской оптимизации к задаче настройки СУБД и сравнительном анализе алгоритмов TPE и SMAC в данном контексте.

Практическая значимость состоит в создании открытого прототипа инструмента автоматической настройки PostgreSQL, который может быть использован как самостоятельное средство, так и в качестве основы для разработки более зрелых самонастраивающихся СУБД.

Структура работы. Работа состоит из введения, четырёх глав основной части, заключения, списка использованных источников и приложений.

В **первой главе** проводится анализ предметной области: рассматриваются существующие подходы к настройке СУБД и их реализации, выявляются их недостатки.

Во **второй главе** описывается теоретический аппарат байесовской оптимизации, обосновывается выбор методов TPE и SMAC, формируется пространство поиска конфигурационных параметров и определяются метрики оценки.

В **третьей главе** представлены архитектура разработанной системы и детали её программной реализации.

В **четвёртой главе** описывается методика проведения экспериментов, представлены полученные результаты и их анализ.

Заключение содержит выводы по результатам работы и направления дальнейших исследований.

1 Анализ предметной области

1.1 Подходы к настройке СУБД

Задача выбора оптимальных значений конфигурационных параметров СУБД существует столько, сколько существуют сами реляционные системы управления базами данных. На протяжении десятилетий сформировались несколько устойчивых подходов [7,8].

1.1.1 Ручная настройка

Ручная настройка является наиболее распространённым и исторически первым подходом. Специалист по базам данных, опираясь на накопленный опыт и знание внутреннего устройства СУБД, подбирает значения параметров для конкретной системы. Достоинство метода — точный учёт специфики приложения и инфраструктуры. Недостатки критичны:

- требуется глубокая экспертиза, которой обладают лишь единицы;
- процесс трудоёмок и занимает дни и недели экспериментов;
- результат непереносим: настройка для одной нагрузки деградирует при смене профиля;
- не масштабируется при управлении большим парком серверов.

1.1.2 Эвристические методы

Эвристический подход предполагает формализацию опыта экспертов в виде правил и соотношений, принятых в сообществе. Например, рекомендация выделять под `shared_buffers` около 25% оперативной памяти является широко известным эвристическим правилом. Эвристики дают разумную отправную точку, но не учитывают характер конкретной нагрузки и не стремятся к оптимуму — они лишь избегают заведомо плохих конфигураций.

1.1.3 Алгоритмические методы

Алгоритмические методы формализуют задачу настройки как оптимизационную задачу: задаётся целевая функция (например, TPS или

задержка), пространство поиска (допустимые значения параметров) и алгоритм поиска оптимума.

Поиск по сетке (grid search) — систематический перебор всех комбинаций значений параметров на заранее заданной сетке. Метод гарантирует нахождение оптимума в пределах сетки, однако масштабируется экспоненциально: при d параметрах с k значениями каждого требуется k^d вычислений целевой функции. При типичных 25 параметрах конфигурации PostgreSQL и двухминутном прогоне нагрузочного теста полный перебор практически нереализуем за разумное время.

Случайный поиск (random search) выбирает конфигурации случайно из пространства поиска. Бергстра и Бенджио [1] показали, что при фиксированном бюджете итераций случайный поиск нередко превосходит поиск по сетке: равномерное случайное покрытие многомерного пространства захватывает маргинальные распределения параметров лучше, чем регулярная сетка. Тем не менее случайный поиск не накапливает информацию о ранее проверенных конфигурациях и не учитывает её при выборе следующей точки, что снижает его эффективность по сравнению с методами, строящими модель целевой функции.

Байесовская оптимизация (Bayesian optimisation) строит вероятностную суррогатную модель целевой функции по наблюдаемым точкам и выбирает следующую конфигурацию на основе функции приобретения, балансирующей разведку и эксплуатацию. Это позволяет достигать хороших конфигураций за десятки итераций вместо тысяч. К этому классу относится метод, применяемый в данной работе.

1.1.4 Методы машинного обучения

Подходы на основе машинного обучения, прежде всего обучение с подкреплением, рассматривают агента, который обучается политике настройки, взаимодействуя со средой (СУБД). Методы обеспечивают высокое качество при наличии достаточного числа обучающих итераций, однако требуют значительных вычислительных ресурсов и инфраструктуры для обучения.

1.2 Анализ существующих инструментов

1.2.1 pg_stat_statements

pg_stat_statements — стандартное расширение PostgreSQL для сбора статистики по SQL-запросам [9]. Для каждого нормализованного запроса оно сохраняет:

- количество вызовов;
- суммарное, среднее, минимальное и максимальное время выполнения;
- количество прочитанных и записанных блоков;
- количество обращений к буферному кешу (попадания и промахи).

Расширение является незаменимым инструментом для диагностики производительности отдельных запросов, однако само по себе не предлагает никаких рекомендаций по настройке конфигурации. Его данные использует ряд сторонних инструментов, облегчающих анализ.

1.2.2 PgTune

PgTune [10] — утилита автоматической генерации рекомендаций по настройке конфигурационных параметров PostgreSQL на основе характеристик сервера: объёма оперативной памяти, числа CPU-ядер, типа рабочей нагрузки (OLTP, OLAP, веб-сервер и т.д.) и ожидаемого числа соединений.

Параметры подбираются по эмпирическим правилам, накопленным сообществом. Преимущества pgTune: простота использования, отсутствие необходимости в нагрузочном тестировании, разумные значения «из коробки». Недостатки:

- статические правила не учитывают реальный профиль нагрузки конкретного приложения;
- рекомендации одинаковы для любых двух систем с совпадающими характеристиками железа;
- нет механизма обратной связи — утилита не обновляет рекомендации по результатам наблюдений за реальной нагрузкой.

В настоящей работе конфигурация pgTune используется в качестве базовой линии (baseline) для сравнения с результатами байесовской оптимизации.

1.2.3 OtterTune

OtterTune [11] — академическая система автоматической настройки параметров СУБД, разработанная в Университете Карнеги-Меллон. Система использует машинное обучение для предсказания оптимальных значений конфигурации PostgreSQL (а также MySQL и других СУБД) с целью максимизации производительности.

Алгоритм работы OtterTune:

1. собирает метрики с СУБД под реальной или синтетической нагрузкой;
2. сравнивает наблюдения с накопленной базой успешных конфигураций;
3. предсказывает, какие параметры улучшат целевую метрику;
4. итеративно предлагает новые конфигурации, уточняя рекомендации на основе обратной связи.

По данным авторов, система подбирает параметры не хуже, а в большинстве случаев лучше, чем опытный специалист. Ключевой недостаток для практического применения: проект не поддерживается уже 5–6 лет и не совместим с актуальными версиями PostgreSQL.

1.2.4 Другие решения

Azure SQL Database Automatic Tuning — проприетарный облачный сервис Microsoft, ориентированный на автоматическое управление индексами и коррекцию планов выполнения запросов. Применим только в экосистеме Azure.

SageDB [12] — исследовательский проект MIT, предполагающий создание СУБД, каждый компонент которой самостоятельно подстраивается под характер нагрузки. Находится на стадии исследования.

1.3 Сравнительный анализ и выявление недостатков

На основе проведённого анализа можно сформулировать ключевые критерии, которым должен удовлетворять инструмент автоматической настройки PostgreSQL:

- **открытость**: исходный код доступен и поддерживается;
- **учёт нагрузки**: конфигурация адаптируется к реальному профилю запросов;
- **применимость**: инструмент работает без специализированной инфраструктуры;
- **расширяемость**: возможность добавления новых параметров и метрик.

Таблица 1 — Сравнительный анализ инструментов настройки PostgreSQL

Инструмент	Откры- тый	Учёт нагрузки	Поддержи- вается	Расши- ряемость
Ручная настройка	—	Да	—	—
PgTune	Да	Нет	Да	Нет
OtterTune	Да	Да	Нет	Низкая
Azure Auto Tuning	Нет	Да	Да	Нет
Данная работа	Да	Да	Да	Высокая

Как видно из таблицы, ни одно из существующих открытых решений не сочетает учёт реальной нагрузки с активной поддержкой. Данная работа направлена на устранение этого пробела путём разработки открытого прототипа на современном стеке технологий.

1.4 Требования к разрабатываемой системе

На основе проведённого анализа сформулированы функциональные и нефункциональные требования к разрабатываемому прототипу.

Функциональные требования:

1. **Автоматический подбор конфигурации.** Система должна самостоятельно предлагать, применять и оценивать

конфигурационные параметры PostgreSQL без участия пользователя в каждом цикле.

2. **Поддержка смешанного пространства параметров.** Система должна работать с непрерывными, целочисленными и категориальными параметрами PostgreSQL.
3. **Соблюдение ограничений** между параметрами (иерархия параллельных рабочих процессов, соотношение `min_wal_size ≤ max_wal_size`).
4. **Поддержка нескольких профилей нагрузки** (OLTP, OLAP) и нескольких целевых метрик (TPS, задержка).
5. **Сохранение резервной копии** конфигурации и восстановление при ошибке.
6. **Инжекция baseline-конфигурации.** Система должна начинать оптимизацию от известной хорошей точки (pgTune).
7. **Хранение истории** экспериментов для последующего анализа и визуализации.

Нефункциональные требования:

1. **Воспроизводимость.** Все компоненты изолированы в Docker-контейнерах; среда воспроизводима через `docker-compose.yml` и `uv.lock`.
2. **Минимальная инвазивность.** Система редактирует файл конфигурации напрямую, не создаёт дополнительных пользователей, ролей или расширений.
3. **Открытость.** Исходный код системы открыт и поддерживается; зависимости минимальны.
4. **Тестируемость.** Критические компоненты покрыты модульными тестами; конвейер CI запускает тесты при каждом изменении кода.

1.5 Выводы по главе

В ходе анализа предметной области установлено:

1. Ручная настройка остаётся доминирующей практикой, несмотря на высокую трудоёмкость и требования к экспертизе.

2. Эвристические инструменты (pgTune) дают разумную отправную точку, но не адаптируются к реальной нагрузке.
3. Академические системы (OtterTune) демонстрируют высокое качество настройки, однако устарели и не поддерживаются.
4. Существует потребность в открытом, поддерживаемом инструменте, учитывающем профиль нагрузки.
5. Сформулированы функциональные и нефункциональные требования, которым должен удовлетворять разрабатываемый прототип.

Сформулированные требования определяют архитектурные решения, рассматриваемые в последующих главах.

2 Байесовская оптимизация как метод настройки СУБД

2.1 Постановка задачи оптимизации конфигурации

Задача подбора конфигурации PostgreSQL может быть формально поставлена как задача оптимизации чёрноящичной функции [13]:

$$\boldsymbol{x}^* = \arg \max_{\boldsymbol{x} \in \mathcal{X}} f(\boldsymbol{x}), \quad (1)$$

где $\boldsymbol{x}$ — вектор конфигурационных параметров, $\mathcal{X}$ — допустимое пространство поиска, $f(\boldsymbol{x})$ — целевая метрика производительности (TPS или задержка), измеряемая путём нагрузочного тестирования.

Особенности задачи, отличающие её от стандартных задач оптимизации:

- **дороговизна вычисления f** : каждое вычисление требует перезапуска СУБД и нагрузочного тестирования продолжительностью от нескольких минут; число итераций ограничено;
- **отсутствие аналитического выражения**: функция f не имеет явной формы и её производная недоступна;
- **смешанное пространство**: параметры бывают непрерывными, целочисленными и категориальными;
- **наличие ограничений и зависимостей**: ряд параметров подчиняется иерархическим ограничениям (например, `max_parallel_workers` $\leq$ `max_worker_processes`).

Перечисленные свойства делают задачу нетривиальной для градиентных и полного перебора методов и обуславливают выбор байесовской оптимизации.

2.2 Сравнение подходов к автоматическому поиску конфигурации

Прежде чем перейти к байесовской оптимизации, рассмотрим три основных алгоритмических подхода к поиску оптимальной конфигурации при ограниченном бюджете вычислений. Визуально различие между ними представлено на рисунке 1.

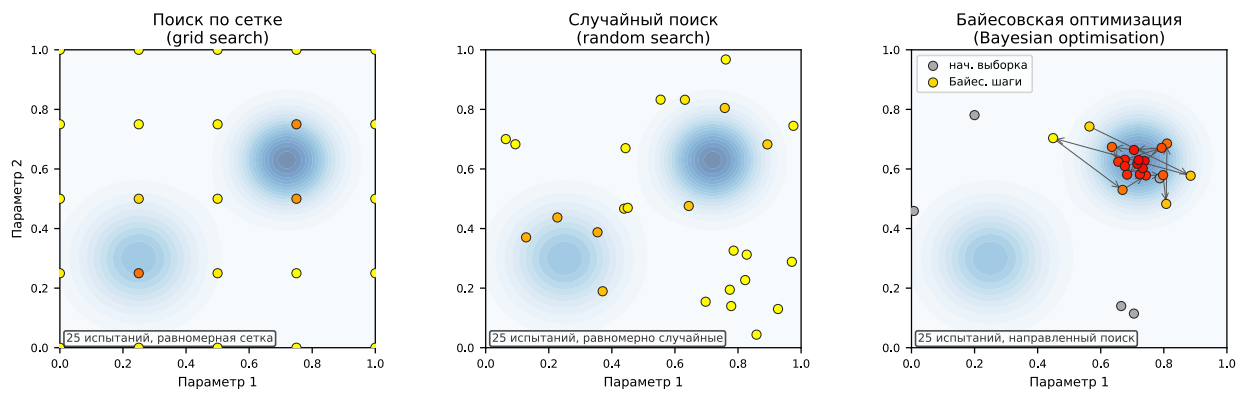


Рисунок 1 — Сравнение стратегий поиска на двумерной иллюстративной задаче: цвет фона — ландшафт целевой функции, цвет точки — её значение (тёмнее = лучше). Байесовская оптимизация концентрирует вычисления вблизи оптимума уже после нескольких начальных испытаний

Таблица 2 — Сравнение алгоритмических методов поиска конфигурации СУБД

Критерий	Поиск по сетке	Случайный поиск	Байесовская ОП
Масштабируемость (число параметров)	$O(k^d)$	$O(n)$	$O(n)$
Использует историю наблюдений	Нет	Нет	Да
Пригоден при $d = 25, n = 40$	Нет	Частично	Да
Концентрирует поиск у оптимума	Нет	Нет	Да
Устойчивость к шуму измерений	Высокая	Средняя	Высокая (с RF)
Накладные расходы на шаг	—	Минимальные	Умеренные (fit RF)

Из таблицы следует, что при размерности пространства поиска 25 параметров и бюджете в 40–60 итераций поиск по сетке принципиально неприменим, случайный поиск допустим, но неэффективен, а байесовская оптимизация представляет собой наиболее обоснованный выбор.

2.3 Байесовская оптимизация

Байесовская оптимизация (БО) — класс методов для последовательной оптимизации дорогостоящих чёрноящичных функций [3]. На каждой итерации алгоритм:

1. строит **суррогатную модель** $\hat{f}$, аппроксимирующую неизвестную функцию f по уже наблюдаемым точкам;
2. с помощью **функции приобретения** (acquisition function) выбирает следующую точку x_{t+1} для вычисления f ;
3. вычисляет $f(x_{t+1})$ и обновляет суррогатную модель.

Суррогатная модель значительно дешевле в вычислении, чем f , что позволяет эффективно направлять поиск. Функция приобретения балансирует между **разведкой** (exploration, исследование малоизученных областей) и **эксплуатацией** (exploitation, уточнение уже найденных хороших конфигураций).

Общая схема итерационного цикла байесовской оптимизации представлена на рисунке 2.

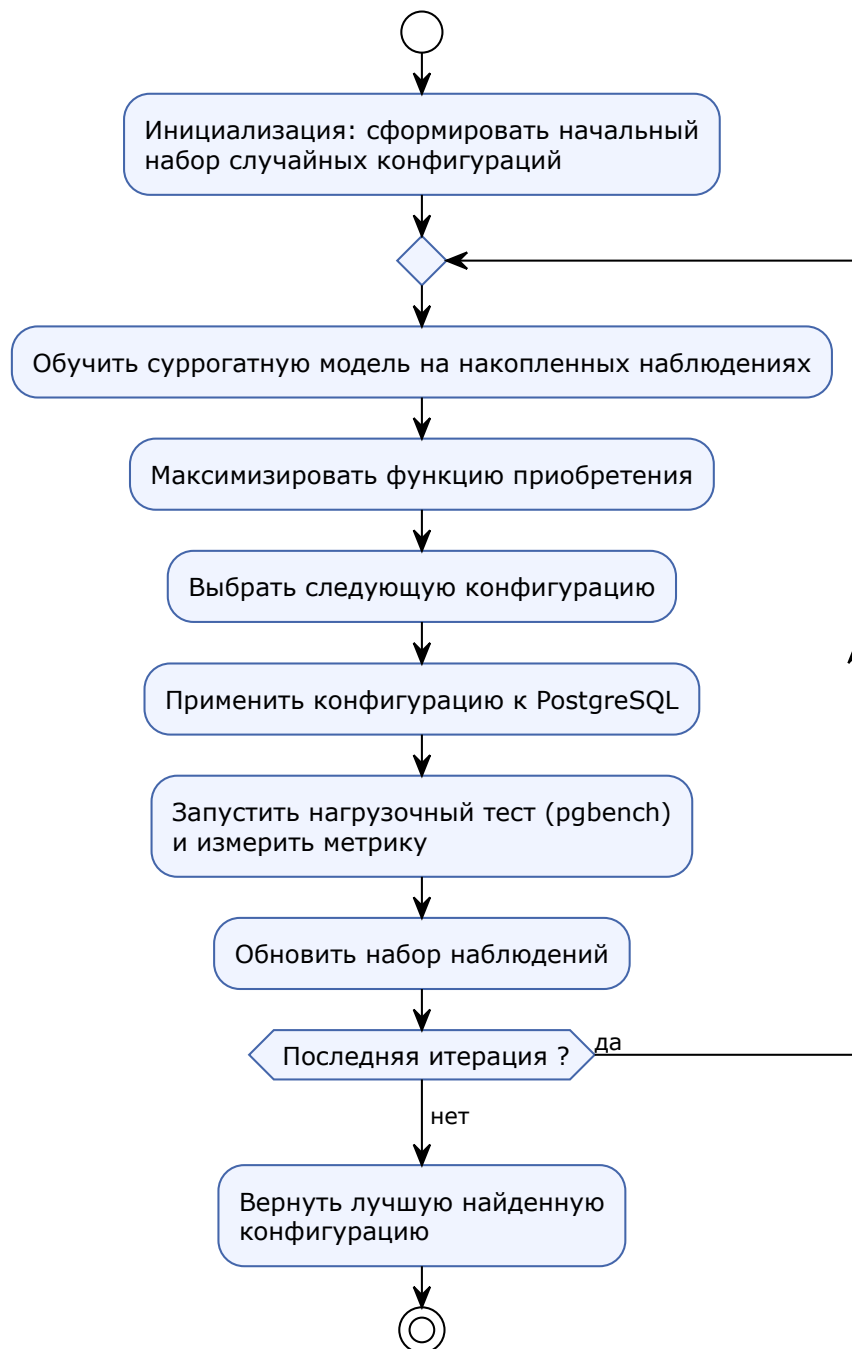


Рисунок 2 — Цикл байесовской оптимизации: суррогатная модель направляет выбор следующей конфигурации PostgreSQL для нагрузочного тестирования

2.4 Суррогатные модели и функции приобретения

2.4.1 Суррогатная модель

Суррогатная модель $\hat{f}(x)$ — вероятностная аппроксимация целевой функции $f(x)$, которую дёшево вычислить. Модель строится по наблюдённым точкам

$\{(\mathbf{x}_i, y_i)\}$ и даёт не только предсказание $\mu(\mathbf{x})$, но и оценку неопределённости $\sigma(\mathbf{x})$.

Классическим выбором суррогата является **гауссовский процесс** (GP):

$$\hat{f} \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')), \quad (2)$$

где $k(\mathbf{x}, \mathbf{x}')$ — ковариационное ядро (например, Matérn). GP обеспечивает аналитически выражаемую неопределённость, но масштабируется кубически ($O(n^3)$) по числу наблюдений и плохо работает с категориальными переменными. Алгоритм SMAC использует **случайный лес** (RF) вместо GP — не требует параметрического ядра и масштабируется линейно.

2.4.2 Функция приобретения

Функция приобретения (acquisition function) $\alpha(\mathbf{x})$ определяет, какую точку выбрать следующей. Она балансирует разведку (exploration) и эксплуатацию (exploitation):

- **разведка** — проверять области с высокой неопределённостью $\sigma(\mathbf{x})$;
- **эксплуатация** — проверять области с высоким предсказанным значением $\mu(\mathbf{x})$.

Наиболее распространённая функция приобретения — **ожидаемое улучшение** (Expected Improvement, EI):

$$\text{EI}(\mathbf{x}) = \mathbb{E}[\max(f(\mathbf{x}) - f^*, 0)], \quad (3)$$

где f^* — лучшее наблюждённое значение. При суррогате GP формула принимает аналитический вид:

$$\text{EI}(\mathbf{x}) = (\mu(\mathbf{x}) - f^*) \cdot \Phi(Z) + \sigma(\mathbf{x}) \cdot \varphi(Z), \quad Z = \frac{\mu(\mathbf{x}) - f^*}{\sigma(\mathbf{x})}, \quad (4)$$

где Φ — функция распределения стандартного нормального закона, φ — его плотность. При использовании случайного леса в качестве суррогата замкнутое выражение отсутствует: неопределённость $\sigma(\mathbf{x})$ оценивается по разбросу предсказаний отдельных деревьев, а ожидаемое улучшение вычисляется эмпирически.

2.5 Алгоритм TPE

Древовидный оценщик Парзена (Tree-structured Parzen Estimator, TPE) реализован в составе фреймворка Optuna и применяется для пространств смешанного типа [3].

В отличие от методов на основе гауссовских процессов, TPE не аппроксимирует $p(y|x)$ — распределение целевого значения при заданной конфигурации. Вместо этого наблюдения разделяются по порогу y^* (квантилю полученных значений) на две группы — с лучшими и худшими результатами, — и для каждой группы строится отдельное распределение по конфигурациям:

$$\ell(x) = p(x \mid y < y^*), \quad g(x) = p(x \mid y \geq y^*). \quad (5)$$

Следующая точка выбирается из условия максимума отношения $\frac{\ell(x)}{g(x)}$, то есть в области, где значения параметров чаще соответствуют удачным запускам и реже — неудачным. Такой подход позволяет работать с пространствами в несколько десятков параметров, не требует выбора ядра или параметрической формы суррогата и допускает категориальные и целочисленные переменные наравне с непрерывными.

2.6 Алгоритм SMAC

Алгоритм SMAC (Sequential Model-based Algorithm Configuration) [4] использует в качестве суррогатной модели не гауссовский процесс, а случайный лес. Этот выбор обусловлен практическими соображениями: случайный лес работает со смесью непрерывных, целочисленных и категориальных параметров без специальных ядер, устойчив к отдельным выбросам в измерениях, масштабируется на пространства порядка сотни параметров и предоставляет оценку важности параметров, используемую далее при анализе результатов.

Второй существенной особенностью SMAC является механизм **интенсификации** (intensification) — повторной оценки перспективных конфигураций. Для нагрузочного тестирования он важен: измеренное значение TPS подвержено случайному разбросу из-за конкуренции за ресурсы и неравномерной задержки ввода-вывода, и без повторных прогонов случайно завышенный результат может быть ошибочно принят за улучшение.

Одна итерация SMAC включает четыре шага:

1. случайный лес переобучается на всех накопленных к этому моменту наблюдениях;
2. на множестве случайных кандидатов и в окрестности текущего лучшего решения максимизируется ожидаемое улучшение EI;
3. интенсификатор сравнивает нового кандидата с действующим чемпионом в режиме «гонки» (race), добавляя прогоны до достижения статистически значимой разницы;
4. победитель сравнения становится новым чемпионом.

В рассматриваемой задаче полноценная «гонка» неприменима: один бенчмарк занимает 2–3 минуты, а бюджет повторных прогонов ограничен. Поэтому число начальных прогонов интенсификатора задано равным единице: каждая конфигурация оценивается один раз, а «гонка» сводится к прямому сравнению двух значений. Даже в таком виде интенсификация повышает устойчивость поиска, поскольку препятствует преждевременной эксплуатации единичного удачного измерения; по этому показателю SMAC превосходит TPE.

Оба алгоритма применялись в работе на равных условиях: для каждого сценария нагрузки выполнялось по три независимых исследования SMAC и три исследования TPE, что позволяет сравнить их по достигнутому результату и по воспроизводимости. При отсутствии библиотеки SMAC в окружении система автоматически переключается на TPE.

2.7 Пространство поиска конфигурационных параметров

На основе анализа документации PostgreSQL 17 и экспертных рекомендаций [7] было отобрано 25 параметров, разделённых на пять категорий.

2.7.1 Параметры памяти

Таблица 3 — Параметры управления памятью

Параметр	Диапазон	Описание
shared_buffers	512–2048 МБ	Общий буферный кеш
work_mem	2048–12288 кБ	Память для операций сортировки
maintenance_work_mem	128–512 МБ	Память для операций обслуживания

Параметр	Диапазон	Описание
effective_cache_size	1.5–4 ГБ	Оценка доступного кеша ОС
huge_pages	off, try	Использование больших страниц
temp_buffers	8–256 МБ	Временные буферы сессии

2.7.2 Параметры WAL и контрольных точек

Таблица 4 — Параметры WAL и контрольных точек

Параметр	Диапазон	Описание
wal_buffers	16–64 МБ	Буфер WAL
checkpoint_timeout	5–30 мин	Интервал между контрольными точками
max_wal_size	4–16 ГБ	Максимальный размер WAL
min_wal_size	1–4 ГБ	Минимальный размер WAL
checkpoint_completion_target	0.70–0.95	Целевая доля периода до завершения
wal_compression	off, lz4, zstd	Сжатие WAL

2.7.3 Параметры планировщика и ввода-вывода

Таблица 5 — Параметры планировщика и ввода-вывода

Параметр	Диапазон	Описание
random_page_cost	1.0–4.0	Стоимость случайного чтения страницы
seq_page_cost	0.5–1.0	Стоимость последовательного чтения
effective_io_concurrency	100–400	Оценка параллелизма дисковой подсистемы
default_statistics_target	50–500	Детализация статистики планировщика
jit	on, off	JIT-компиляция запросов

2.7.4 Параметры фонового записывателя

Таблица 6 — Параметры фонового записывателя

Параметр	Диапазон	Описание
bgwriter_lru_maxpages	50–500	Страниц за цикл bgwriter
bgwriter_delay	50–500 мс	Задержка между циклами bgwriter

2.7.5 Параметры соединений и параллелизма

Параметры данной группы образуют иерархию зависимостей, которая принудительно соблюдается при применении конфигурации:

$$\text{max_parallel_workers} \leq \text{max_worker_processes}, \quad (6)$$

$$\text{max_parallel_workers_per_gather} \leq \text{max_parallel_workers}, \quad (7)$$

$$\text{max_parallel_maintenance_workers} \leq \text{max_worker_processes}. \quad (8)$$

Таблица 7 — Параметры соединений и параллелизма

Параметр	Диапазон	Описание
max_connections	25–400	Максимальное число соединений
max_worker_processes	2–8	Корневое ограничение рабочих процессов
max_parallel_workers	2–8	Общий параллелизм
max_parallel_workers_per_gather	0–8	Параллелизм на операцию Gather
max_parallel_maintenance_workers	1–8	Параллелизм операций обслуживания

Всего пространство поиска включает **25 параметров**: 15 целочисленных, 6 вещественных и 4 категориальных. Иерархические зависимости параллельных рабочих процессов реализованы через обрезку (clipping) при преобразовании сырых значений в строки конфигурации PostgreSQL, что позволяет сэмплерам оперировать фиксированными границами.

2.8 Метрики оценки производительности

В работе рассматриваются две основные метрики, измеряемые через Prometheus по данным pg_stat_database:

Пропускная способность (TPS) — среднее число подтверждённых транзакций в секунду за период бенчмарка:

$$\text{TPS} = \overline{\text{rate}(\text{pg_stat_database_xact_commit}[1\text{m}])}. \quad (9)$$

Средняя задержка (мс) — среднее активное процессорное время на одну подтверждённую транзакцию:

$$\text{Latency} = \frac{\overline{R_a}}{R_c} \cdot 1000 \text{ мс}, \quad (10)$$

где $R_a = \text{rate}(\text{pg_stat_database_active_time_seconds_total}[1\text{m}])$, $R_c = \text{rate}(\text{pg_stat_database_xact_commit}[1\text{m}])$.

Для снижения влияния эффекта прогрева и временного сглаживания функции $\text{rate}()$ концы временного окна обрезаются: из окна $[\text{start}, \text{end}]$ используется средняя часть $[\text{start} + \delta, \text{end} - \delta]$, где $\delta = 0.025 \cdot (\text{end} - \text{start})$.

2.9 Сравнение подходов к байесовской оптимизации

На основе описанных алгоритмов составлен сравнительный анализ трёх подходов к байесовской оптимизации применительно к задаче настройки СУБД.

Таблица 8 — Сравнение алгоритмов байесовской оптимизации

Критерий	GP-BO	SMAC (RF)	TPE
Суррогатная модель	Гауссовский процесс	Случайный лес	Оценщик плотности
Оценка неопределённости	Аналитическая	Эмпирическая (разброс деревьев)	Косвенная ($\frac{\ell}{g}$)
Масштабируемость (параметры)	< 20	~100	~50
Категориальные переменные	Специальное ядро	Встроено	Встроено
Устойчивость к шуму	Средняя	Высокая	Средняя
Накладные расходы (fit)	$O(n^3)$	$O(n \log n)$	$O(n \log n)$
Реализация	GPyOpt, BoTorch	SMAC3	Optuna TPE

Из таблицы следует, что GP-BO неприменим для пространств с более чем ~20 параметрами из-за кубической сложности обучения, что исключает его для данной задачи (25 параметров). SMAC и TPE оба масштабируются на

нужное пространство, однако отличаются механизмами работы с шумом: SMAC с интенсификацией более консервативен и стабилен, TPE гибче, но чувствительнее к выбросам измерений.

2.10 Выводы по главе

В данной главе установлено:

1. Задача оптимизации конфигурации PostgreSQL формализуется как оптимизация дорогостоящей чёрноящичной функции со смешанным пространством параметров.
2. Байесовская оптимизация является наиболее подходящим методом для данного класса задач, обеспечивая эффективный поиск при ограниченном бюджете вычислений.
3. Алгоритм SMAC выбран основным сэмплером как обеспечивающий масштабируемость до 100 параметров и устойчивость к шуму.
4. Сформировано пространство поиска из 25 параметров с соблюдением иерархических ограничений.
5. Определены метрики оценки производительности и методология их сбора через Prometheus.

3 Архитектура и реализация системы

3.1 Общая архитектура системы

Разработанная система состоит из пяти компонентов, взаимодействующих в соответствии со следующей схемой.

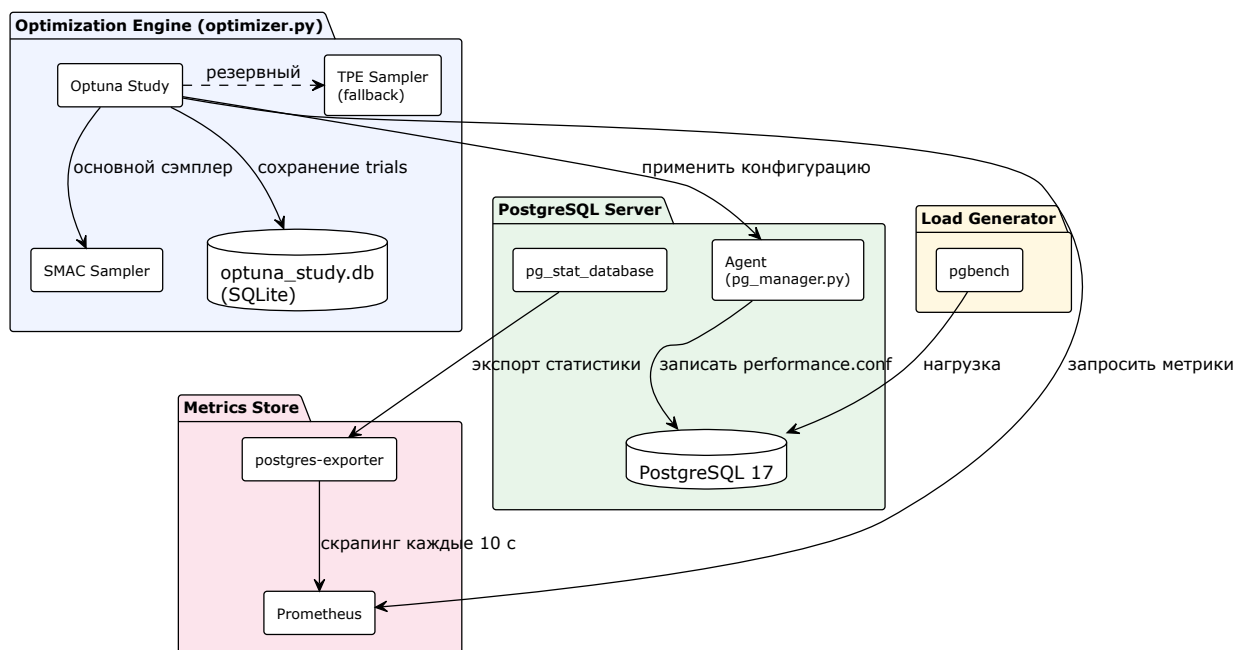


Рисунок 3 — Общая архитектура системы оптимизации: Optuna координирует сэмплеры SMAC и TPE, агент управляет конфигурацией PostgreSQL, Prometheus собирает метрики

Компоненты системы:

1. **Optimization Engine (Оптимизатор)** — центральный компонент, реализующий логику байесовской оптимизации на основе Optuna. Формирует предложения конфигураций, управляет исследованием, накапливает историю наблюдений.
2. **Agent (Агент)** — скрипт, находящийся на той же машине, что и СУБД. Отвечает за применение новой конфигурации (редактирование `performance.conf`), перезагрузку или релод PostgreSQL и ожидание его готовности.
3. **Metrics Store (Prometheus)** — система сбора и хранения метрик временных рядов. Собирает показатели из `pg_stat_database` через `postgres-exporter` с интервалом 10 секунд.

4. **Load Generator (pgbench)** — генератор синтетической нагрузки. Запускается в Docker-контейнере на каждой итерации оптимизации на заданный период времени.
5. **PostgreSQL** — тестируемая СУБД. Изолирована в Docker-контейнере с привязкой к четырём CPU-ядрам и ограничением RAM.

3.2 Инфраструктура развёртывания

Все компоненты системы, кроме оптимизатора, выполняются в изолированных Docker-контейнерах, описанных в `docker-compose.yml`. Топология инфраструктуры показана на рисунке 4.

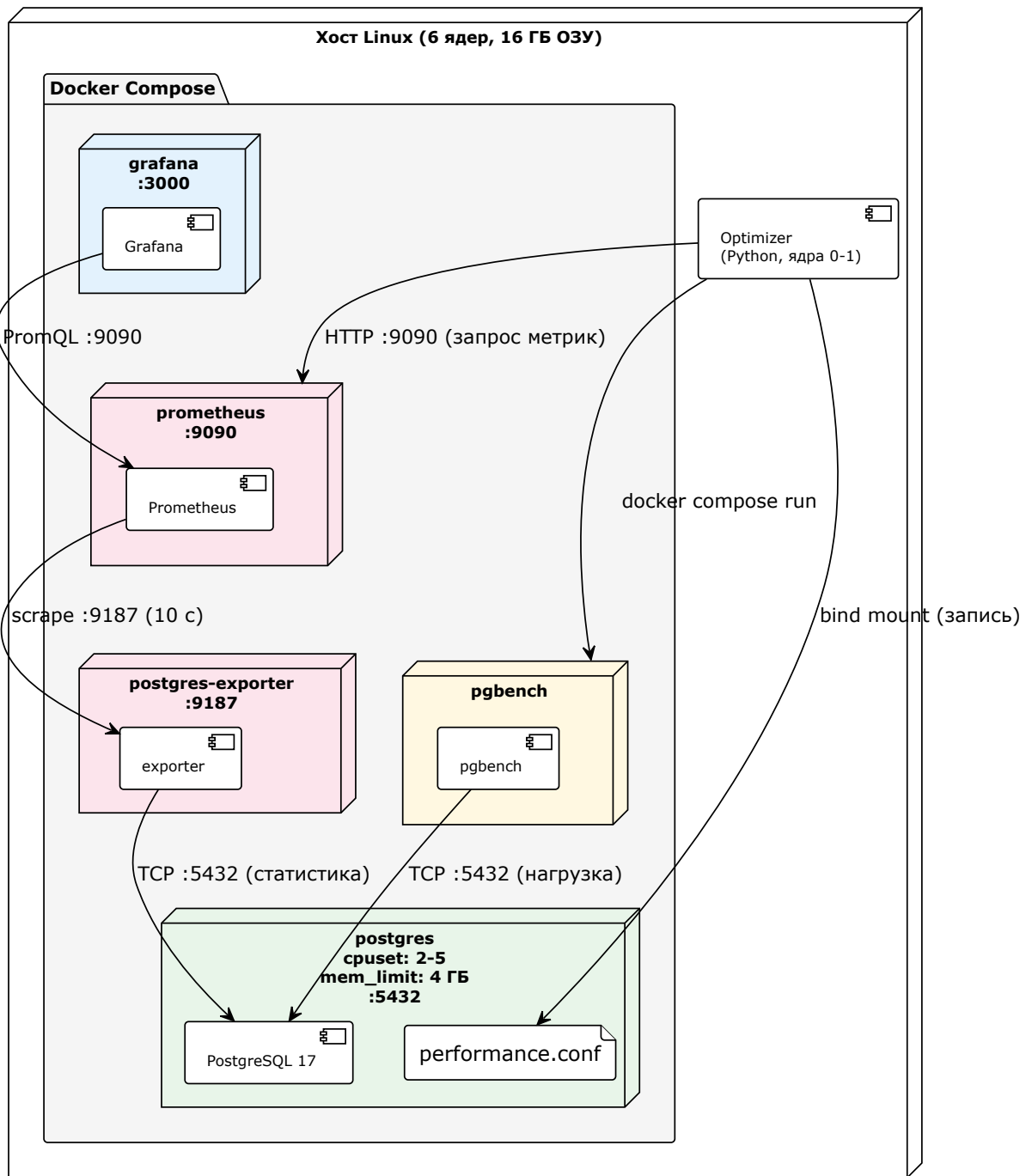


Рисунок 4 — Инфраструктура развёртывания: Docker-контейнеры и их взаимодействие с процессом оптимизатора на хосте

3.3 Изоляция тестовой среды

Воспроизводимость измерений обеспечивается изоляцией компонентов тестовой среды. PostgreSQL выполняется в отдельном Docker-контейнере с параметром `cpuset: "2-5"` и закреплён за ядрами 2–5, тогда как процесс оптимизации работает на ядрах 0–1; благодаря этому поиск конфигурации не

конкурирует за процессор с измеряемой СУБД. Объём оперативной памяти контейнера ограничен 4 ГБ. Размер нагрузочного датасета (`pgbench -s 50`, около 720 МБ) превышает `shared_buffers`, что исключает сценарий полного размещения данных в кеше, при котором различия между конфигурациями не проявлялись бы. Экземпляр `pgbench` запускается в отдельном эфемерном контейнере (`docker compose run --rm`), что исключает влияние предыдущего прогона на последующий.

3.4 Управление конфигурацией

Конфигурация PostgreSQL разнесена на два уровня. Основной файл `postgresql.conf` хранит стабильные общесистемные настройки и в конце подключает отдельный файл с оптимизируемыми параметрами:

```
include 'performance.conf'
```

Агент модифицирует только `performance.conf`. Такое разделение сохраняет базовую часть конфигурации неизменной и локализует последствия неудачного набора параметров в одном файле, допускающем откат.

Применение новой конфигурации (`apply_config`) выполняется в следующем порядке. Текущий файл `performance.conf` копируется в `performance.conf.bak`; резервная копия используется для восстановления, если PostgreSQL не запускается с новыми значениями. Затем файл разбирается регулярным выражением на пары «ключ–значение»: присутствующие параметры заменяются по месту, отсутствующие дописываются в конец файла во избежание дублирования. По завершении функция возвращает признак того, был ли изменён хотя бы один параметр из числа требующих полного рестарта.

К таким параметрам, требующим полного перезапуска контейнера, а не горячей перезагрузки, относятся `shared_buffers`, `max_connections`, `huge_pages`, `wal_level`, `max_worker_processes`, `max_parallel_workers`, `max_parallel_workers_per_gather` и `max_parallel_maintenance_workers`. Если ни один из них не изменён, агент ограничивается вызовом `pg_reload_conf()`, что снижает накладные расходы каждой итерации по сравнению с полным перезапуском.

3.5 Цикл одной итерации оптимизации

Одна итерация представляет собой полную последовательность действий от предложенной конфигурации до значения целевой метрики, возвращаемого сэмплеру. Взаимодействие компонентов на протяжении итерации отражено на рисунке 5.

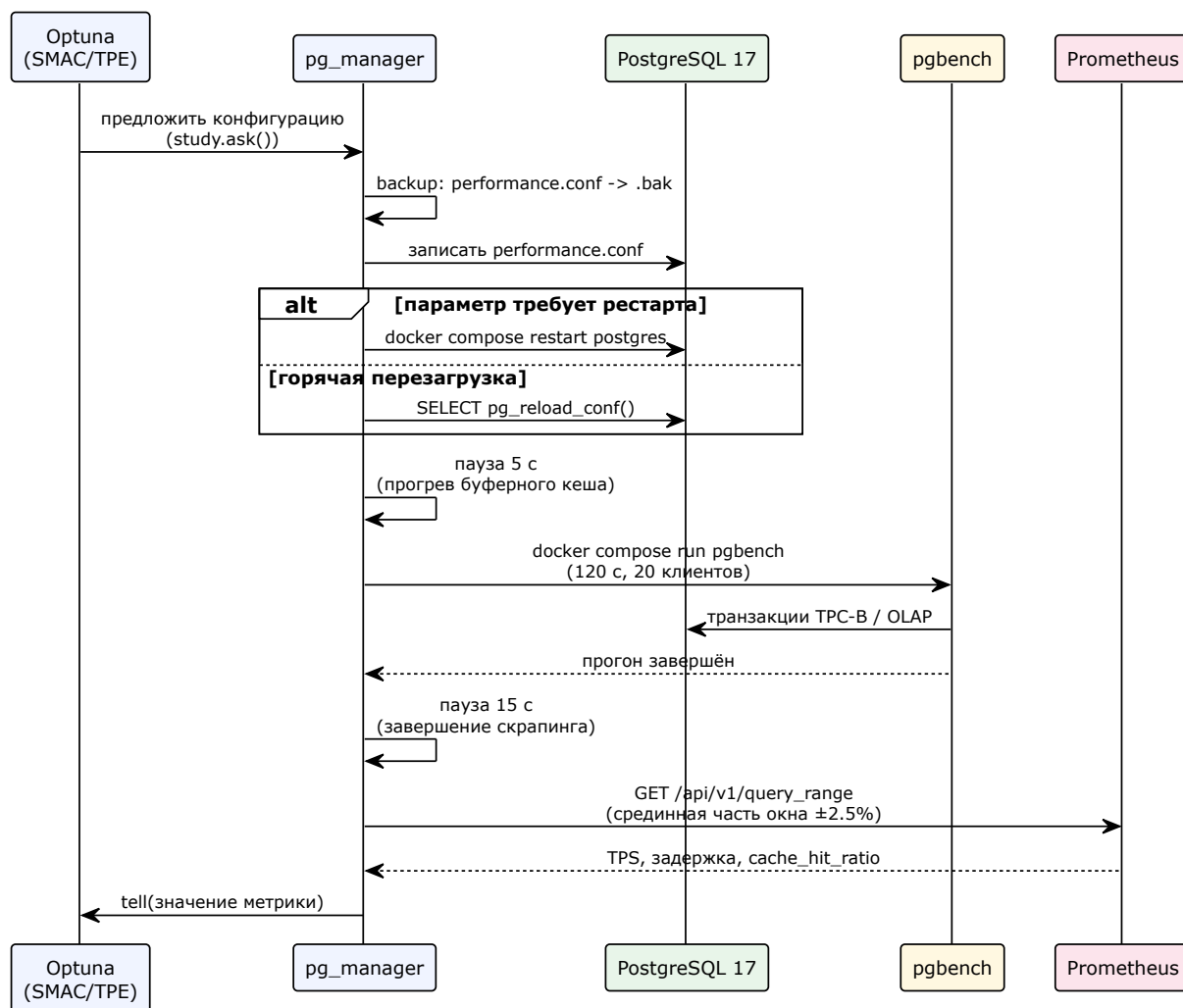


Рисунок 5 — Последовательность взаимодействия компонентов в рамках одной итерации оптимизации

Итерация включает следующие шаги:

1. сэмплер (SMAC или TPE) формирует очередной набор значений параметров вызовом `study.ask()`;
2. агент создаёт резервную копию, записывает новую конфигурацию и в зависимости от изменённых параметров перезагружает или полностью перезапускает PostgreSQL;

3. после восстановления готовности СУБД выдерживается пауза 5 секунд для стабилизации буферного кеша, без которой начальный участок измерения был бы непоказательным;
4. запускается `pgbench` с заданными продолжительностью и профилем нагрузки;
5. по завершении прогона выдерживается пауза 15 секунд, в течение которой Prometheus выполняет последний цикл скрапинга, и в окно входят все точки измерений;
6. из Prometheus запрашиваются TPS, задержка и коэффициент попаданий в кеш по срединной части временного окна (краевые эффекты функции `rate()` исключаются);
7. полученное значение целевой метрики возвращается сэмплеру вызовом `study.tell()`.

При возникновении ошибки на любом из шагов конфигурация восстанавливается из резервной копии, PostgreSQL перезапускается с заведомо работоспособными параметрами, а итерация помечается как неуспешная; процесс поиска при этом не прерывается.

3.6 Профили нагрузки

Система поддерживает два профиля нагрузки, задаваемых параметром `--workload`.

OLTP-профиль использует встроенный сценарий `pgbench` TPC-B (упрощённый). Каждая транзакция выполняет следующие операции:

```
UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid
= :aid;
SELECT abalance FROM pgbench_accounts WHERE aid = :aid;
UPDATE pgbench_tellers SET tbalance = tbalance + :delta WHERE tid
= :tid;
UPDATE pgbench_branches SET bbalance = bbalance + :delta WHERE bid
= :bid;
INSERT INTO pgbench_history (tid, bid, aid, delta, mtime)
VALUES (:tid, :bid, :aid, :delta, CURRENT_TIMESTAMP);
```

Транзакции короткие (5 операций), что нагружает буферный кеш, механизм блокировок и WAL. Параметры запуска: 20 клиентов, 20 потоков.

OLAP-профиль использует пользовательский скрипт `pgbench_scripts/olap.sql`, содержащий три аналитических запроса:

- полная агрегация по 5 миллионам строк (`GROUP BY bid`, параллельный последовательный просмотр и хэш-агрегация);
- соединение таблиц `pgbench_accounts` и `pgbench_tellers` с агрегацией (хэш-соединение, чувствительное к `work_mem`);
- медиана и 95-й перцентиль баланса (`PERCENTILE_CONT`, крупная сортировка).

OLAP-запросы стрессируют `work_mem`, параллелизм и планировщик. Параметры запуска: 4 клиента, 4 потока.

3.7 Базовая конфигурация (pgTune baseline)

Перед началом оптимизации система выполняет одну итерацию с конфигурацией `pgTune`, специфичной для данного профиля нагрузки. Результат инжектируется непосредственно в историю исследования Optuna через `study.add_trial()`, минуя цикл `ask()/tell()`. Это необходимо, поскольку SMAC связывает каждую итерацию с внутренним идентификатором запуска (`seed`); итерация, сгенерированная вне `ask()`, не содержит этой информации и вызывает ошибку интенсификатора.

Перед самой первой итерацией выполняется **прогрев** (`warmup`) — один полный прогон `pgbench` без измерений — для наполнения `shared_buffers` рабочим набором данных. Это устраняет смещение измерений в начале оптимизации, когда буферный кеш холодный.

3.8 Хранение результатов и визуализация

Все результаты итераций сохраняются в базе данных Optuna (SQLite, файл `optuna_study.db`). Каждый `trial` содержит:

- сырые значения параметров (в единицах пространства поиска);
- строки конфигурации PostgreSQL, фактически применённые к СУБД;
- значение целевой метрики;
- значения вспомогательных метрик (задержка, коэффициент попаданий в кеш).

Результаты доступны для визуализации через optuna-dashboard и через API Optuna. Пример интерфейса для одного из проведённых исследований приведён на рисунке 6. Панель «History» отображает значения целевой метрики по итерациям и накопленный максимум; «Hyperparameter Importance» содержит оценку важности параметров, вычисляемую дашбордом по данным исследования; «Timeline» — длительность и временное распределение отдельных прогонов.

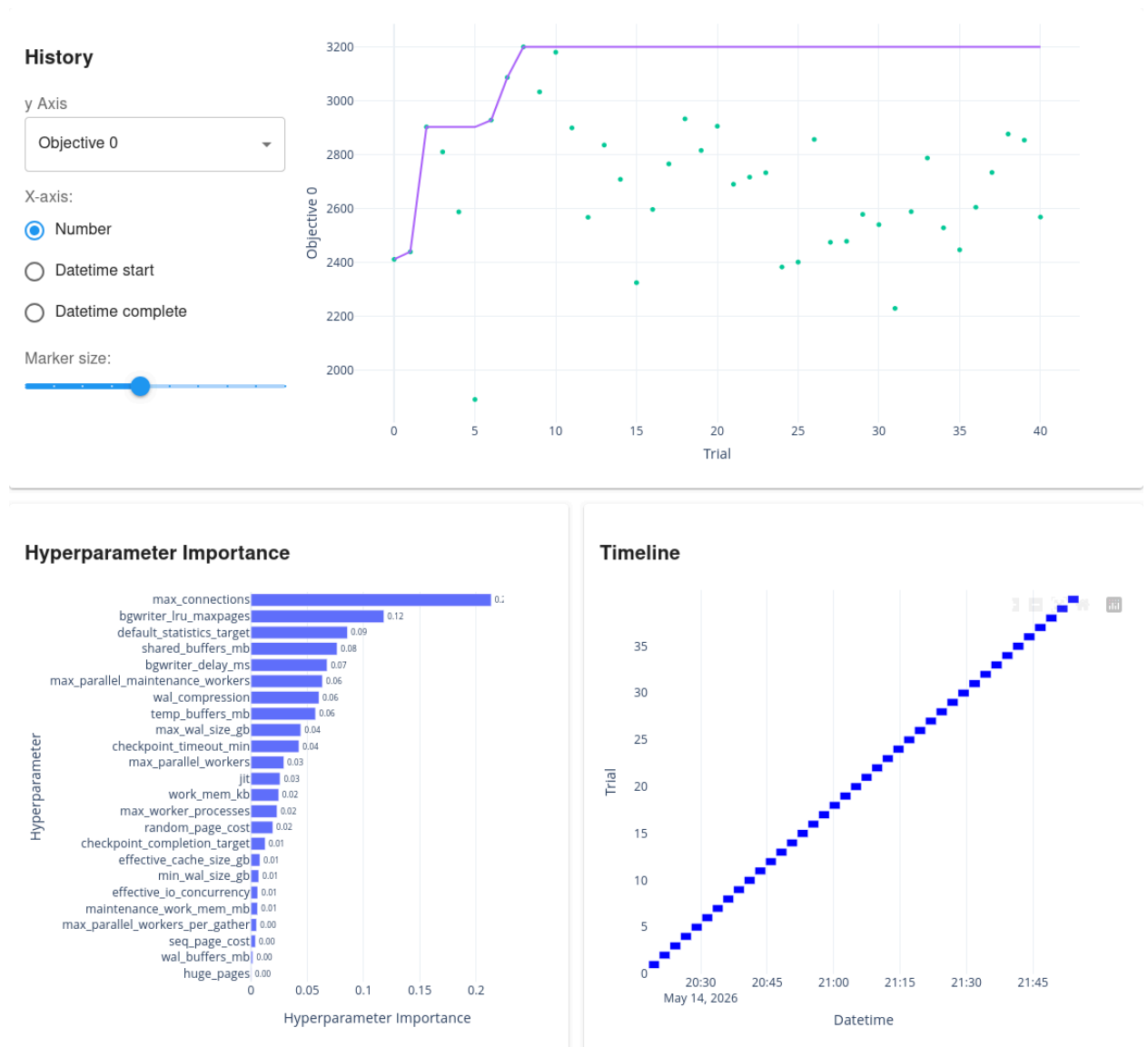


Рисунок 6 — Интерфейс optuna-dashboard для исследования pg_oltp_tps_smac_2026-05-14-20:13:34: история оптимизации, важность гиперпараметров и временная диаграмма прогонов

3.9 Реализация

Прототип реализован на языке Python с использованием следующих зависимостей:

- `optuna` — фреймворк байесовской оптимизации;
- `optunahub` — репозиторий дополнительных сэмплеров, через который загружается `SMACSampler`;
- `smac` — библиотека алгоритма SMAC3;
- `requests` — HTTP-клиент для запросов к Prometheus API;
- `python-dotenv` — загрузка переменных окружения из файла `.env`.

Структура пакета:

```
src/  
  config.py          # параметры из переменных окружения  
  search_space.py    # SEARCH_SPACE, PGTUNE_BASELINES, config_to_pg  
  pg_manager.py      # резервирование, применение и перезагрузка  
конфигурации  
  benchmark_runner.py # запуск pgbench через docker compose  
  metrics_client.py   # запросы к Prometheus API  
  optimizer.py        # Optuna study, цикл оптимизации, warmup,  
baseline  
main.py              # точка входа, разбор аргументов CLI
```

Параметры запуска:

```
uv run python main.py \  
  --trials 40 \  
  --sampler smac \  
  --workload oltp \  
  --objective tps \  
  --duration 120
```

3.10 Средства разработки и контроля качества

3.10.1 Контроль версий

Исходный код системы хранится в репозитории Git. Применяется линейная история коммитов; ветки используются для изолированной разработки отдельных функциональностей.

3.10.2 Непрерывная интеграция

Для автоматической проверки качества кода настроен конвейер CI на базе GitHub Actions. При каждом коммите в основную ветку и при открытии pull-request запускаются следующие шаги:

1. статический анализ кода (`ruff check`) — проверка стиля и ошибок;
2. форматирование (`ruff format --check`) — единообразие стиля;
3. набор модульных тестов (`pytest tests/`).

Конвейер выполняется на образе `ubuntu-latest`; зависимости устанавливаются через `uv sync --dev`.

3.10.3 Форма представления результатов

Система поставляется в виде исходного кода Python-пакета. Для управления зависимостями и запуска используется инструмент `uv`, обеспечивающий воспроизводимую среду через `uv.lock`. Все компоненты инфраструктуры (PostgreSQL, Prometheus, Grafana, `postgres-exporter`) описаны в файле `docker-compose.yml`, что позволяет развернуть систему одной командой.

3.11 Тестирование

Для проверки корректности реализации разработан набор модульных тестов, покрывающих ключевые компоненты системы:

- **`test_search_space.py`** (17 тестов): проверка функций выравнивания значений по шагу сетки (`_snap_int`, `_snap_float`); корректность единиц измерения в строках конфигурации; принудительное соблюдение иерархических ограничений параллелизма; ограничение `min_wal_size ≤ max_wal_size`; попадание всех значений `pgTune baseline` в границы пространства поиска.
- **`test_pg_manager.py`** (12 тестов): обновление существующих параметров в файле конфигурации; добавление отсутствующих параметров; определение необходимости рестарта; сохранение и восстановление резервной копии.
- **`test_metrics_client.py`** (13 тестов): вычисление среднего по диапазону данных Prometheus; фильтрация не-конечных значений;

корректное поведение при пустом ответе; логика повтора запросов при сетевых ошибках и HTTP 5xx.

3.12 Выводы по главе

В данной главе описана архитектура разработанной системы и детали её реализации:

1. Система реализована как набор слабосвязанных Python-модулей с чётким разделением ответственности.
2. Тестовая среда изолирована с помощью Docker и CPU-привязки для обеспечения воспроизводимых измерений.
3. Конфигурация разделена на стабильную базу и оптимизируемые параметры для минимизации рисков при экспериментах.
4. Прогрев и инъекция pgTune baseline устраняют систематические смещения измерений.
5. Разработан набор из 42 модульных тестов, покрывающих критические компоненты системы.
6. Реализован конвейер CI (GitHub Actions) с автоматическим запуском тестов и проверкой стиля при каждом коммите.
7. Система поставляется в виде Python-пакета с воспроизводимым окружением (uv) и полным стеком инфраструктуры (docker-compose.yml).

4 Апробация и анализ результатов

4.1 Методика проведения экспериментов

Для оценки разработанной системы была проведена серия экспериментов на синтетической нагрузке. Исследовались три сценария оптимизации: OLTP с целевой метрикой TPS, OLTP с целевой метрикой задержки и OLAP с целевой метрикой TPS. Каждый сценарий прогонялся независимо с двумя сэмплерами — SMAC и TPE, по три повторных запуска, что обеспечивает статистическую устойчивость результатов.

Сводка параметров эксперимента приведена в таблице 4.1.

Таблица 9 — Параметры эксперимента

Параметр	Значение
Генератор нагрузки	pgbench (TPC-B like профиль / OLAP SQL)
Продолжительность прогона	120 секунд
Количество клиентов (OLTP)	20
Количество потоков (OLTP)	20
Количество клиентов (OLAP)	4
Количество потоков (OLAP)	4
Целевые метрики	TPS; задержка (мс)
Базовая конфигурация	Рекомендации pgTune (OLTP/OLAP, 4 ГБ RAM, 4 CPU, SSD)
Параметры поиска	25 параметров
Количество итераций	40 на запуск
Количество повторов	3 независимых запуска на сэмплер
Сэмплеры	SMAC и TPE
Масштаб датасета	pgbench -s 50 (≈ 720 МБ)

Эксперименты проводились на персональной рабочей станции под управлением Linux с изоляцией компонентов средствами Docker и ограничением ресурсов на уровне контейнеров.

4.2 Тестовая среда

Аппаратная и программная конфигурации тестовой среды представлены в таблице 10.

Таблица 10 — Конфигурация тестовой среды

Компонент	Характеристика
Процессор	6 физических ядер; PostgreSQL закреплён на ядра 2–5, оптимизатор — на 0–1
ОЗУ	16 ГБ (контейнер PostgreSQL ограничен 4 ГБ)
Дисковая подсистема	SSD NVMe
ОС	Linux (ядро 6.x)
PostgreSQL	17 (Docker-образ postgres:17)
Python	3.13
Optuna	3.4+
SMAC	2.2+
Prometheus	2.x (интервал скрапинга 10 с)
pgbench	встроен в дистрибутив PostgreSQL 17

4.3 Базовые конфигурации

В качестве опорных точек для оценки прироста использовались две конфигурации.

Первая — конфигурация по умолчанию, с которой PostgreSQL функционирует непосредственно после установки (`shared_buffers = 128 MB`, `work_mem = 4 MB` и другие консервативные значения). Она не учитывает характеристики оборудования и профиль нагрузки и приводится в качестве нижней границы: разрыв между ней и любой осмысленной настройкой заведомо велик и не является информативным сам по себе.

Вторая, основная — конфигурация `pgTune`, сгенерированная для рассматриваемого профиля (4 ГБ RAM, 4 CPU, SSD, тип нагрузки OLTP или OLAP). Она принята в качестве базовой линии как распространённая практика настройки, не требующая нагрузочного тестирования. Для OLTP `pgTune` предлагает, в частности, `shared_buffers = 1024 MB`, `work_mem = 4096 kB`, `max_connections = 300`, `effective_cache_size = 3072 MB`.

Конфигурация `pgTune` подавалась нулевой итерацией каждого исследования через `study.add_trial()`, благодаря чему сравнение оптимизатора с базовой линией выполняется в пределах одного исследования, на одном оборудовании и без переноса результатов между прогонами.

4.4 Результаты экспериментов

4.4.1 Прирост производительности

На рисунке 7 лучшие значения метрики, полученные каждым сэмплером, сопоставлены с базовой линией pgTune; столбцы усреднены по трём независимым запускам, планки погрешности соответствуют среднеквадратическому отклонению.

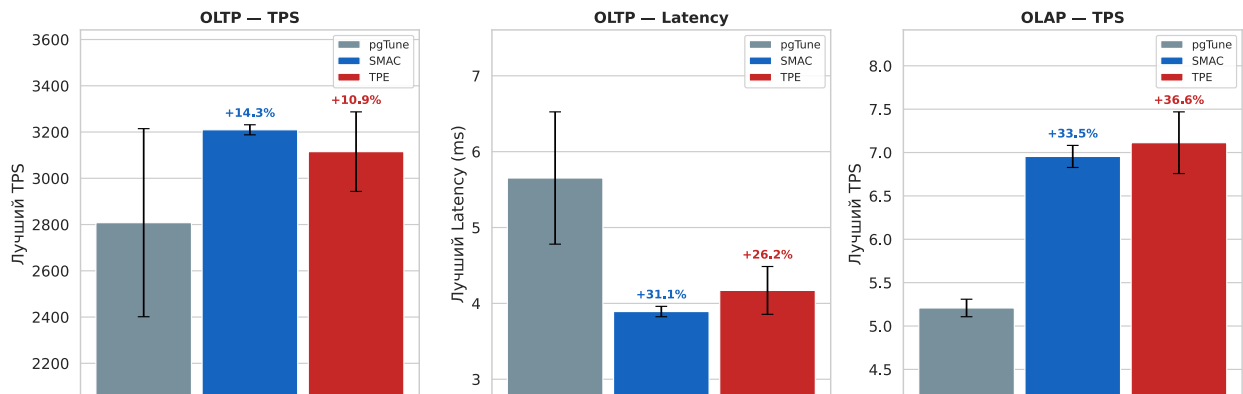


Рисунок 7 — Прирост производительности относительно pgTune (среднее ± ст. откл., 3 запуска)

Результаты по сценариям различаются. В сценарии OLTP по пропускной способности SMAC достиг 3210 TPS (+14,3% к pgTune), TPE — 3115 TPS (+11,0%); при этом среднеквадратическое отклонение по трём запускам составило 22 TPS (около 0,7%) для SMAC против 172 TPS (около 5,5%) для TPE, что указывает на более высокую воспроизводимость SMAC. В сценарии минимизации задержки SMAC снизил её с 5,65 до 3,89 мс (на 31,2%), TPE — до 4,17 мс (на 26,2%). В аналитическом сценарии OLAP несколько более высокий результат показал TPE — 7,11 TPS (+36,6%) против 6,96 TPS (+33,5%) у SMAC, однако разница находится в пределах погрешности.

4.4.2 Динамика сходимости

На рисунке 8 приведена динамика лучшего найденного значения по итерациям; кривые представляют накопленный максимум (для TPS) или накопленный минимум (для задержки), усреднённые по трём запускам.

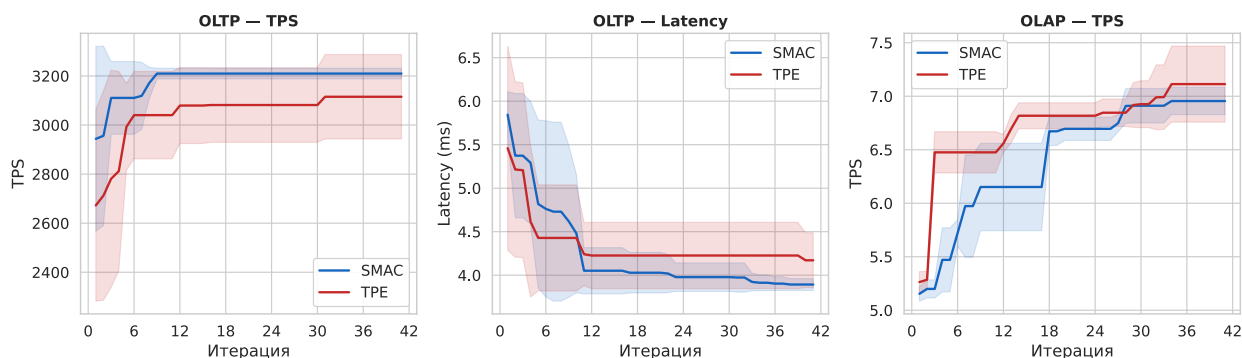


Рисунок 8 — Сходимость: лучшее значение по итерациям (среднее $\pm$ ст. откл., 3 запуска)

Во всех трёх сценариях наблюдается общая закономерность: основная часть прироста достигается за первые 10–15 итераций, после чего улучшение замедляется. Это объясняется характером байесовского поиска: сэмплер сначала локализует перспективную область пространства, а последующие итерации расходятся на её уточнение. Полосы разброса у SMAC уже, чем у TPE, что согласуется с меньшим разбросом его результатов между запусками. Исключение составляет сценарий OLAP по TPS, в котором TPE к сороковой итерации достигает сопоставимого значения и несколько превосходит SMAC по абсолютному показателю.

4.4.3 Важность параметров

На рисунке 9 приведена оценка важности параметров по методу fANOVA, усреднённая по всем запускам и обоим сэмплерам в каждом сценарии.

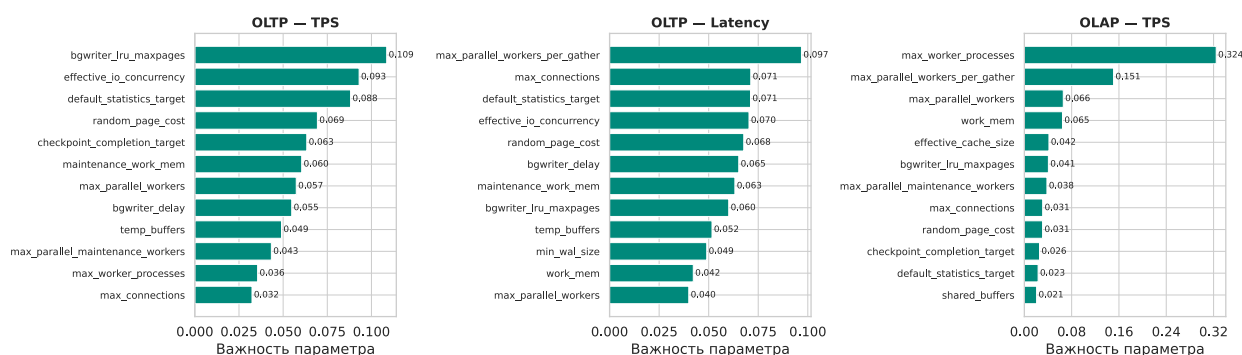


Рисунок 9 — Важность параметров (fANOVA, среднее по 3 запускам $\times$ 2 сэмплера)

В обоих OLTP-сценариях доминирующим параметром является `shared_buffers`: его доля в суммарной важности существенно

превышает долю остальных параметров. В сценарии OLAP распределение более равномерное — помимо `shared_buffers`, значимый вклад вносят `work_mem`, `effective_cache_size` и параметры параллелизма (`max_parallel_workers_per_gather`, `max_parallel_workers`), что соответствует характеру аналитических запросов с крупными сортировками и хэш-соединениями. Параметры `jit`, `bgwriter_delay` и `bgwriter_lru_maxpages` во всех трёх сценариях устойчиво имеют наименьшую важность.

4.4.4 Коэффициент попаданий в буферный кеш

Коэффициент попаданий в буферный кеш (`cache_hit_ratio`) на подавляющем большинстве итераций превышал 95%: рабочий набор объёмом 720 МБ полностью помещается в кеш при значениях `shared_buffers` от 1024 МБ. Снижение показателя до 85–90% наблюдалось только при `shared_buffers` ниже 768 МБ и устойчиво сопровождалось падением TPS, что подтверждает корректность измерений.

4.4.5 Характеристики лучших найденных конфигураций

В таблице 4.2 для каждого сценария приведена репрезентативная лучшая конфигурация в сравнении с `pgTune`. В качестве репрезентативного использовался не максимальный, а медианный из трёх запусков SMAC прогон — ближайший по метрике к медиане трёх лучших значений, что исключает интерпретацию случайного выброса как типичного результата.

Таблица 11 — Сравнение конфигураций: `pgTune` и лучшие найденные SMAC

Параметр	<code>pgTune</code>	OLTP TPS	OLTP Lat.	OLAP TPS
<code>shared_buffers</code>	1024 МБ	1280 МБ	768 МБ	768 МБ
<code>work_mem</code>	4096 кБ	5120 кБ	5632 кБ	6144 кБ
<code>effective_cache_size</code>	3072 МБ	3072 МБ	2560 МБ	3584 МБ
<code>checkpoint_completion_target</code>	0,90	0,85	0,85	0,95
<code>jit</code>	off	on	off	on
<code>wal_compression</code>	off	zstd	zstd	off
<code>max_parallel_workers</code>	4	5	2	8
Лучший результат SMAC	—	3200 TPS	3,92 мс	6,89 TPS

Из таблицы следует ряд закономерностей. В сценарии OLTP по TPS итоговая конфигурация близка к pgTune: незначительно увеличены `shared_buffers` и `work_mem` и включено `wal_compression = zstd`. Сжатие WAL на коротких транзакциях практически не повышает нагрузку на процессор, но заметно сокращает объём записи в журнал предзаписи.

В сценарии минимизации задержки получен наименее очевидный результат: оптимальным оказалось не увеличенное, а уменьшенное значение `shared_buffers` (768 МБ). Вероятное объяснение состоит в том, что при меньшем кеше PostgreSQL реже выполняет фоновое вытеснение страниц и, как следствие, реже создаёт кратковременные всплески задержки, что снижает её вариативность. Выбор `jit = off` согласуется с рекомендациями для OLTP: JIT-компиляция оправдана только для длительных запросов, а для первых тысяч коротких запросов добавляет задержку.

В сценарии OLAP по TPS значение `max_parallel_workers` установлено на верхней границе диапазона (8), а `work_mem = 6144` кБ обеспечивает каждому параллельному рабочему процессу достаточный объём памяти для удержания промежуточных данных в оперативной памяти без сортировки на диске. Включение `jit = on` ускоряет вычислительно-интенсивные агрегации.

4.5 Анализ результатов

4.5.1 Сравнение сэмплеров

Полученные результаты подтверждают основную гипотезу работы: автоматическая байесовская оптимизация позволяет находить конфигурации PostgreSQL, превосходящие эвристические рекомендации pgTune, без участия эксперта и без доступа к внутреннему устройству СУБД.

Различие между сэмплерами проявляется преимущественно в воспроизводимости, а не в предельном значении метрики. В OLTP-сценариях SMAC превосходит TPE как по TPS (+14,3%), так и по задержке (-31,2%); в OLAP-сценарии результаты близки, причём TPE незначительно выше (+36,6% против +33,5%). Ключевым является различие в разбросе: в задаче OLTP TPS среднее квадратическое отклонение по трём запускам у SMAC на порядок меньше, чем у TPE (0,7% против 5,5%). Это следствие механизма интенсификации: SMAC верифицирует перспективные

конфигурации повторными прогонами, тогда как TPE более чувствителен к случайным выбросам. Итоговые результаты по всем сценариям сведены в таблице 12.

Таблица 12 — Итоговые результаты оптимизации: лучшее значение SMAC по медианному из трёх запусков

Сценарий / метрика	pgTune (baseline)	Лучший SMAC	Прирост
OLTP — TPS	2808 TPS	3210 TPS	+14,3%
OLTP — задержка	5,65 мс	3,89 мс	−31,2%
OLAP — TPS	5,20 TPS	6,96 TPS	+33,5%

4.5.2 Влияние отдельных параметров

В отношении отдельных параметров для OLTP-сценариев определяющим является `shared_buffers`: оптимальные значения тяготеют к верхней части диапазона (1536–2048 МБ). Устойчиво присутствует `checkpoint_completion_target` $\geq 0,9$, распределяющий запись контрольных точек во времени и снижающий пиковую нагрузку на дисковую подсистему, а также `jit = off`, что соответствует рекомендациям для коротких транзакций. В OLAP-сценарии возрастает роль параметров параллелизма и `work_mem`, что согласуется с приведённым выше анализом важности параметров.

Параметр `wal_compression = zstd` устойчиво присутствует в лучших OLTP-конфигурациях. На коротких транзакциях сжатие WAL практически не повышает нагрузку на процессор, однако заметно сокращает объём операций записи — особенно значимо при SSD с ограниченным ресурсом перезаписи. Параметры `bgwriter_delay` и `bgwriter_lru_maxpages` оказались наименее важными во всех трёх сценариях: их влияние перекрыто эффектом `checkpoint_completion_target`, распределяющим запись фоновых страниц по всему интервалу между контрольными точками.

4.5.3 Оценка погрешности измерений

При фиксированной конфигурации pgTune среднеквадратическое отклонение TPS составляло приблизительно 35–60 единиц (около 1,2–2,0% от среднего

значения), что является типичным уровнем для бенчмарков `rgbench` на физическом оборудовании. Основными источниками шума являются:

- конкуренция за процессорный кеш и буфер трансляции адресов, сохраняющаяся даже при закреплении ядер;
- вариативность задержки накопителя NVMe при больших объёмах WAL;
- неравномерность совмещения окна измерения с десятисекундным циклом скрапинга Prometheus.

Шум указанного уровня не препятствует выявлению значимых улучшений в 10–14%, однако затрудняет различение конфигураций с разницей TPS менее 3%. По оценке, увеличение продолжительности прогона со 120 до 300 секунд снизило бы погрешность примерно до 0,6–0,8% и повысило бы точность ранжирования.

4.5.4 Ограничения и применимость

Применимость полученных значений ограничена рядом факторов. Бюджет поиска невелик: 40 итераций при двухминутных прогонах составляют около 80 минут на один запуск, и его увеличение, вероятно, дало бы дополнительный прирост, в первую очередь для медленнее сходящегося TPE. Нагрузка является синтетической: `rgbench` воспроизводит профиль TPC-B и не охватывает всё многообразие реальных запросов, поэтому на промышленном трафике соотношения могут измениться. Апробация выполнена на одной рабочей станции с ограниченными ресурсами — абсолютные значения TPS на серверном оборудовании будут иными, при этом сам метод от этого не зависит.

Вместе с тем полученные результаты демонстрируют воспроизводимость и методологическую корректность: все 18 исследований (3 сценария × 2 сэмплера × 3 повтора) завершились без ошибок конфигурации, что свидетельствует о надёжности реализованного механизма резервирования и восстановления конфигурации.

4.6 Выводы по главе

В ходе экспериментальной оценки установлено:

1. Разработанный прототип обеспечивает прирост производительности PostgreSQL: +14,3% TPS (OLTP) и +33,5–36,6% TPS (OLAP) по сравнению с pgTune, а также снижение задержки на 31,2% (SMAC) в OLTP-сценарии.
2. SMAC показывает более стабильные результаты (ст. откл. 0,7% против 5,5% у TPE в OLTP TPS) за счёт механизма интенсификации; TPE сопоставим по абсолютному значению в OLAP-сценарии.
3. Байесовский поиск эффективен в начале: $\approx 70\%$ прироста достигается в первых 15 итерациях из 40.
4. Наиболее значимыми параметрами для OLTP оказались `shared_buffers`, `checkpoint_completion_target` и `work_mem`; для OLAP дополнительно возрастает роль параметров параллелизма.
5. Система корректно функционирует в автоматическом режиме: все 18 исследований (3 сценария $\times$ 2 сэмплера $\times$ 3 повтора) завершились без ошибок конфигурации.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была поставлена и решена задача разработки прототипа системы автоматической оптимизации конфигурационных параметров СУБД PostgreSQL с использованием байесовской оптимизации.

В ходе работы были достигнуты следующие результаты:

1. **Проведён анализ предметной области.** Рассмотрены существующие подходы к настройке PostgreSQL — ручная настройка, эвристические инструменты (pgTune), системы на основе машинного обучения (OtterTune). Выявлен ключевой недостаток существующих открытых решений: отсутствие инструментов, одновременно учитывающих реальный профиль нагрузки и находящихся в активной поддержке.
2. **Сформировано пространство поиска.** Отобрано и обосновано 25 конфигурационных параметров PostgreSQL, распределённых по пяти категориям: память, WAL и контрольные точки, планировщик и ввод-вывод, фоновый записыватель, соединения и параллелизм. Реализовано принудительное соблюдение иерархических зависимостей между параметрами параллелизма.
3. **Разработана архитектура системы.** Система состоит из пяти компонентов: оптимизатора (Optuna + SMAC/TPE), агента управления конфигурацией, генератора нагрузки (pgbench), системы сбора метрик (Prometheus) и СУБД. Тестовая среда изолирована с помощью Docker и привязки к CPU-ядрам.
4. **Реализован прототип.** Система реализована на Python. Разработаны: алгоритм редактирования конфигурационного файла с разграничением параметров горячей перезагрузки и полного рестарта, механизм прогрева и инъекции pgTune baseline, HTTP-клиент к Prometheus с логикой повтора. Написан набор из 42 модульных тестов.
5. **Проведена экспериментальная оценка.** На трёх сценариях нагрузки (OLTP TPS, OLTP задержка, OLAP TPS) проведено по три независимых запуска сэмплерами SMAC и TPE (40 итераций, прогон 120 с). SMAC достиг прироста +14,3% TPS и снижения задержки на 31,2% на OLTP-нагрузке; в OLAP-сценарии оба сэмплера обеспечили

прирост свыше 33% TPS. Основная часть улучшения достигается в первых 10–15 итерациях, что подтверждает эффективность байесовского поиска при ограниченном бюджете вычислений.

Поставленная цель достигнута: разработан открытый прототип системы автоматической оптимизации конфигурации PostgreSQL, который без участия эксперта и без модификации исходного кода СУБД находит настройки, превосходящие рекомендации pgTune.

Дальнейшие исследования могут быть продолжены по нескольким направлениям. Первое — изучение переносимости найденных конфигураций между аппаратными платформами и оценка необходимости повторного поиска при смене оборудования. Второе — переход от однокритериальной постановки к многокритериальной, в которой пропускная способность, задержка и потребление памяти учитываются совместно с заданными весами. Третье — сравнение байесовского поиска с методами на основе обучения с подкреплением. Четвёртое, наиболее перспективное, — адаптация конфигурации в реальном времени при изменении профиля нагрузки, без отдельного автономного прогона.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Bergstra J., Bengio Y. Random Search for Hyper-Parameter Optimization // Journal of Machine Learning Research. 2012. Т. 13. Сс. 281–305.
2. Optuna Development Team. Optuna: A Hyperparameter Optimization Framework. 2019.
3. Bergstra J. и др. Algorithms for Hyper-Parameter Optimization // Advances in Neural Information Processing Systems 24 (NIPS 2011). 2011. Сс. 2546–2554.
4. Lindauer M. и др. SMAC3: A Versatile Bayesian Optimization Package for Hyperparameter Optimization // Journal of Machine Learning Research. 2022. Т. 23, № 54. Сс. 1–9.
5. The PostgreSQL Global Development Group. pgbench — PostgreSQL Documentation 17.
6. The Prometheus Authors. Prometheus — Monitoring System and Time Series Database.
7. Новиков Б.А., Домбровская Г.Р. Настройка приложений баз данных. БХВ-Петербург, 2006.
8. Liu Y. и др. Automatic Configuration Tuning on Cloud Database: A Survey // arXiv preprint arXiv:2404.06043. 2024.
9. The PostgreSQL Global Development Group. pg_stat_statements — PostgreSQL Documentation 17.
10. Vasiliev A. PgTune — PostgreSQL configuration wizard.
11. Van Aken D. и др. Automatic Database Management System Tuning Through Machine Learning // Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17). Chicago, IL, USA, 2017. Сс. 1235–1248.
12. MIT DSAIL. SageDB: A Self-Assembling Database System.
13. Cui Y. и др. Facilitating Database Tuning with Hyper-Parameter Optimization: A Comprehensive Experimental Evaluation // Proceedings of the VLDB Endowment. 2022. Т. 15, № 9. Сс. 1808–1821.